Indian Institute of Technology Indore

IITISOC 2026

Mid-Way Evaluation Report

Project Title: Universal File Type Converter Web Application

Problem Statement: PS-3

Team Members

Team Leader:

Vidadala Kalyani (250001077)

Vithanala Anushka (250001080)

Kondamuri Tathwik (250021010)

Submission Date: 30/06/2026

Contents

1 Abstract

Format incompatibility is a universal frustration. Users frequently struggle with restrictive paywalls, intrusive advertisements, and data privacy concerns when using online file converters. Our project, the Universal File Type Converter Web Application, addresses these issues by providing a secure, efficient, and ad-free platform for multi-format document and image conversions. Driven by the need for absolute data privacy, our platform implements strict server-side cleanup routines that automatically delete user assets exactly 30 minutes post-conversion. Currently, we have successfully initialized our development environment and mapped backend routes using Express and Multer to safely parse payloads with strict 10MB size limits and file type validations. On the frontend, we have designed the React-based drag-and-drop workspace. Regarding core processing, we have successfully integrated ImageMagick for basic image conversions, while document conversion routes via Pandoc are structurally mapped and pending final execution. Furthermore, our automated security cleanup cron job is fully active and functional.

2 Problem Understanding

2.1 Problem Statement

Users constantly need to convert files between formats (e.g., PDF to DOCX, JPG to PNG) but lack secure, free, and reliable tools. Existing solutions compromise user privacy or gatekeep features behind paywalls. This project demands a full-stack web application with a drag-and-drop interface, real-time feedback, and rigorous backend data deletion protocols to ensure secure format transformations.

2.2 Objectives

- Develop an intuitive, drag-and-drop frontend interface for seamless file uploads.
- Implement real-time status updates reflecting the conversion state (Uploading $\rightarrow$ Processing $\rightarrow$ Ready).
- Integrate server-side logic and external CLI libraries for accurate multi-format conversions.
- Enforce strict server-side security via cron jobs to delete uploaded and converted files after 30 minutes.

2.3 Target Users

The primary target users are students, developers, designers, and office professionals who require frequent and quick document or asset conversions without compromising their sensitive information.

2.4 Applications

- Extracting text from locked PDFs for academic or professional editing.
- Compressing and converting image assets (e.g., JPG to PNG) for web development.
- Managing personal documents securely without leaving digital traces on third-party servers.

3 Existing Solutions

Solution	Strengths	Limitations
Cloud Convert	Supports a massive variety of formats and provides an API.	Heavily restricted free tier; files are kept on servers longer than ideal for strict privacy.
Zamzar	Easy to use, email notifications for completed tasks.	Slow processing speeds for free users; intrusive ads; file size limitations.
Smallpdf	Excellent UI/UX and targeted specifically at document manipulation.	Expensive premium plans; limits free users to a small number of tasks per day.

4 Functional Requirements

4.1 Core Features

- **Multi-Format Processing:** Support for document (PDF, DOCX, TXT) and image (JPG, PNG) conversions.
- **Drag-and-Drop Interface:** Responsive upload zone for quick file selection.
- **Real-Time Feedback:** Progress indicators for user request states.
- **Secure File Handling:** Automated backend cron jobs to purge files within 30 minutes.

4.2 Additional Features

- **User Authentication:** Standard email/password login for accessing personalized "Conversion History".
- **Bulk Conversion:** Support for batch uploading and downloading as a single .zip file.

5 Technology Stack

Component	Technology	Reason
Frontend	React.js, Tailwind	Component-based architecture and rapid UI styling.
Backend	Node.js, Express	Asynchronous event-driven architecture suitable for I/O heavy tasks.
File Parser	Multer	Safely parses multi-part form payloads and enforces file size constraints.
Conversion Utilities	ImageMagick, Pandoc	Robust, open-source CLI tools for image and document transformation.
Task Scheduler	Node-cron	Ideal for programming automated file cleanup tasks.
Deployment	Docker	Containerized workspace ensuring consistency across environments.

6 System Architecture

The system follows a modular client-server architecture. The user interacts with the React frontend to upload files. These multi-part form payloads are sent to the Express.js backend where Multer validates extensions and enforces a strict 10MB size limit. The backend triggers isolated background processes using Pandoc or ImageMagick depending on the MIME type. Processed files are mapped to download links, and a functional Node-cron job continuously monitors the storage path, purging any files older than 30 minutes.

7 Database Design

Though primarily a stateless converter regarding file assets, a lightweight schema is planned to handle User Authentication and Conversion History.

- **Users Table:** `id` (PK), `email`, `password_hash`, `created_at`.
- **History Table:** `id` (PK), `user_id` (FK), `original_name`, `source_format`, `target_format`, `status`, `timestamp`.

8 API Design

Method	Endpoint	Description
POST	`/api/auth/register`	Creates a new user account.
POST	`/api/upload`	Accepts multi-part form data, validates 10MB limits, and saves to temp storage.
POST	`/api/convert`	Triggers the CLI tool for a specific file identifier and target format.
GET	`/api/download/:id`	Retrieves the converted file securely.
GET	`/api/history`	Fetches conversion history for an authenticated user.

9 UI/UX Design

The interface is designed to be minimalistic. The central piece is a large dashed-border drag-and-drop workspace. Below it, file cards appear displaying the file name, an icon, and a dropdown for the target format. Progress bars are strategically placed to indicate upload and conversion phases.

10 Project Modules

Module	Description
Frontend Pipeline	Dashboard layout, drag-and-drop workspace, visual progress bars.
Backend & Routing	Server infrastructure, Multer file upload managers enforcing type/size rules.
Core Conversion	Connection to CLI utilities (ImageMagick/Pandoc), isolated runtime execution.
Security & DevOps	Docker configuration, directory security, automated cleanup cron tasks.

11 Current Progress

11.1 Completed

- Initialized code repository and set up localized Docker development containers.
- Designed and built the React-based drag-and-drop frontend workspace.
- Mapped backend server routes using Express and configured Multer to safely parse payloads with a strict 10MB size limit and file type validations.
- Successfully integrated ImageMagick for basic core image processing.
- Developed and activated the automated security cleanup cron job (fully functional).

11.2 In Progress

- Finalizing document conversion execution via Pandoc (currently structurally mapped).
- Preparing system mid-evaluation working prototype across branches.

11.3 Yet to Start

- Multi-browser responsive check runs and comprehensive end-to-end testing.
- Cloud deployment of the final application.

12 Challenges Faced

- **Challenge 1:** Enforcing safe payload parsing without exceeding memory limits.
  Solution: Implemented a strict 10MB validation barrier via Multer before the stream is fully written to the disk.

- **Challenge 2:** Setting up reliable cross-platform CLI tool execution for document and image manipulations.
  Solution: Isolating system processes and wrapping the environment in Docker ensures dependencies like ImageMagick and Pandoc execute consistently.

13 Future Work

Timeline	Planned Work
July 1 – July 14	Complete final execution of Pandoc document conversion commands; run format input exception protection routines.
July 15 – August 2	Execute multi-browser responsive checks; host final platform on cloud services; prepare final repository and documentation.

14 GitHub Repository

Repository Link: `https://github.com/cse250001077/FileMorph-IITISOC`
Work Distribution: Vidadala Kalyani (Backend/APIs), Vithanala Anushka (Frontend UI), Kondamuri Tathwik (DevOps/Security).
Current Status: The backend core architecture and Docker containerization are successfully deployed on the `main` branch. The React drag-and-drop workspace is being actively managed on the `frontend-dev` branch to safely integrate components before opening Pull Requests for the final evaluation.

15 Testing

- **Unit Testing:** Writing isolated tests for the cron job cleanup logic and 10MB Multer size limit validations.
- **Manual Testing:** Uploading malformed files to verify rejection headers, UI error handling, and testing ImageMagick output integrity.

- **Edge Cases:** Handling concurrent file uploads and checking cron job overlapping states.

16 Screenshots

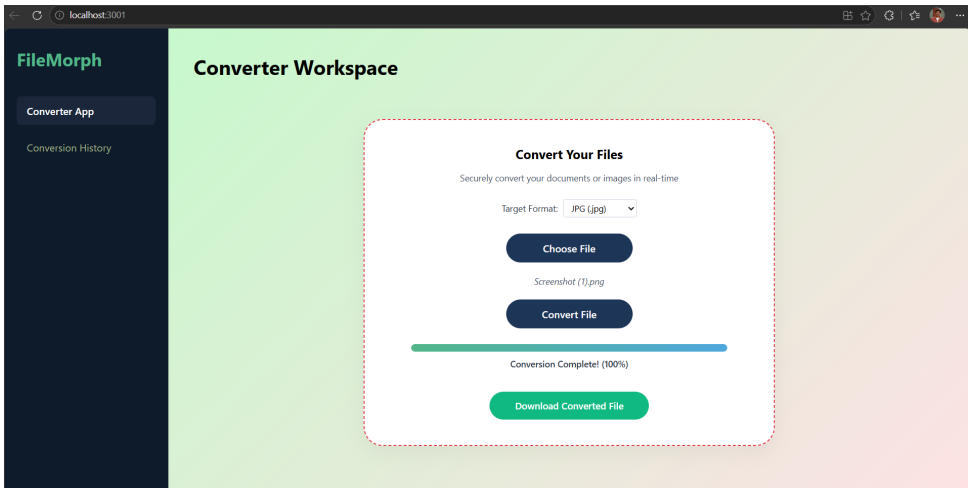

Figure 1: Frontend UI workspace layout

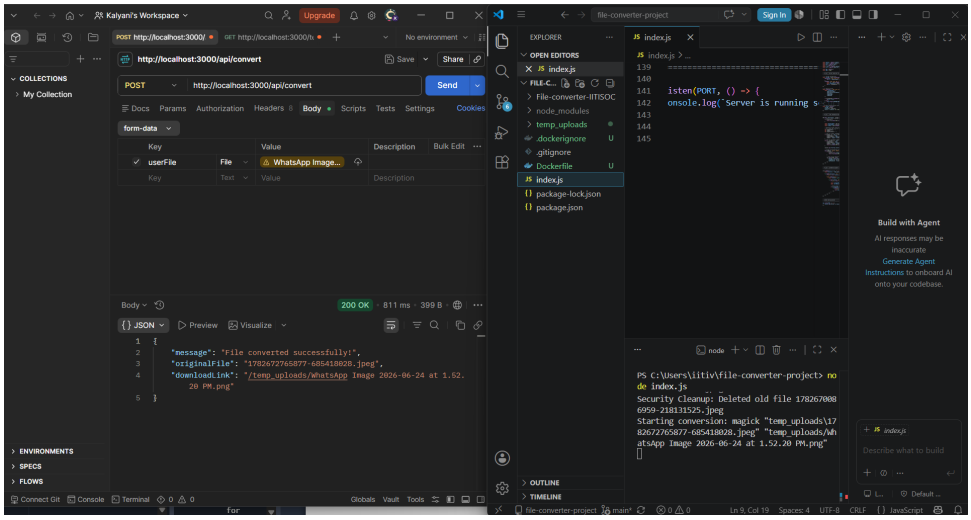

Figure 2: Postman API test verification

17 References

1. Node.js `fs` module documentation.
2. React.js and Tailwind CSS official documentation.
3. ImageMagick and Pandoc CLI guides.
4. Multer and Node-cron npm repository guidelines.
5. Docker documentation for containerized workflows.